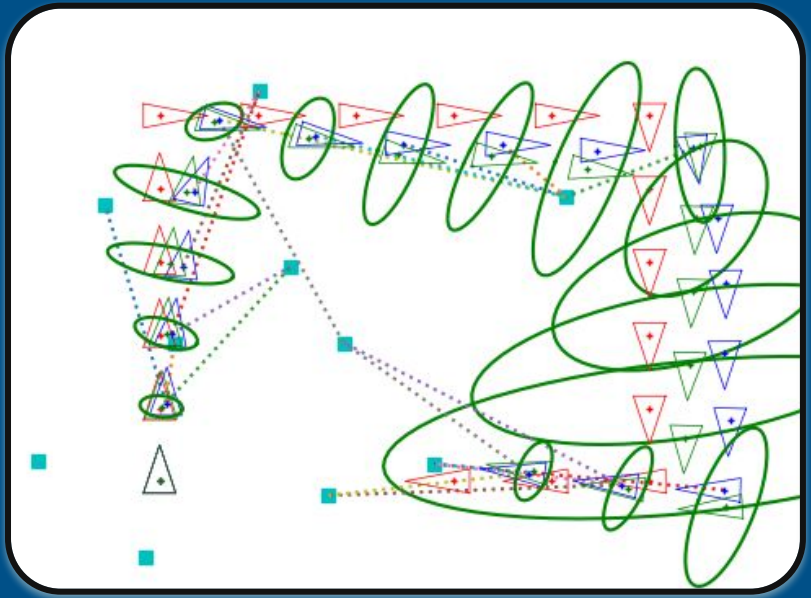
I402: Principios de la Robótica Autónoma

Clases Prácticas
Tutorial 1



Universidad de SanAndrés

¿Quienes somos?

Tadeo Casiraghi

tcasiraghi@udesa.edu.ar

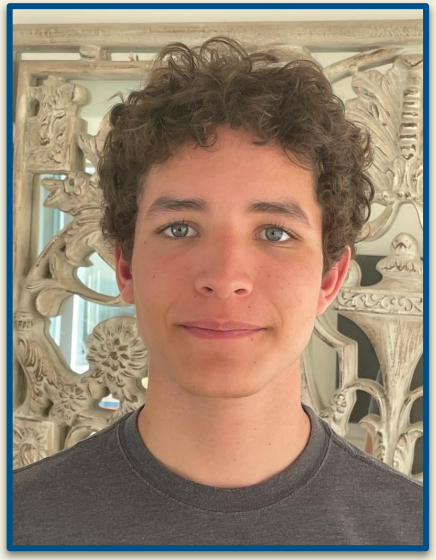
Bautista Chasco

bchasco@udesa.edu.ar

¿Quienes somos?

Lucio Luque Materazzi

lluquematerazzi@udesa.edu.ar



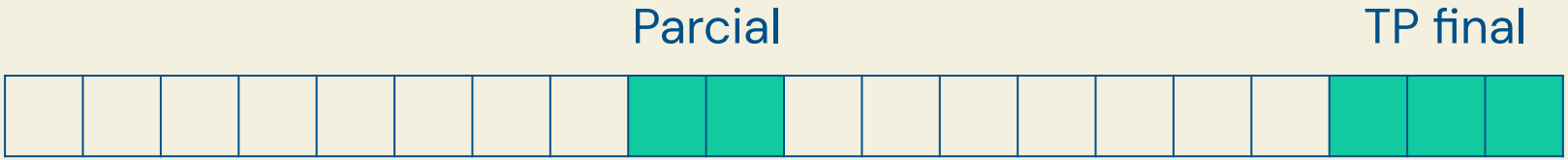
Teo Manuel Kaucher

tkaucher@udesa.edu.ar

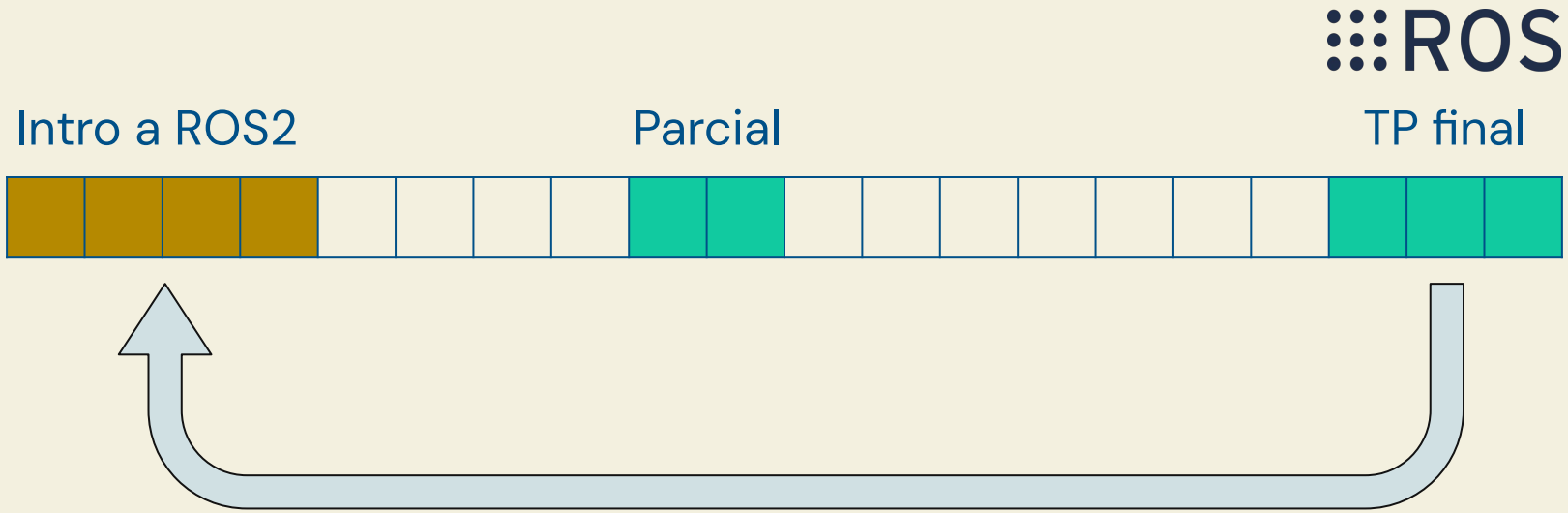
¿Qué vamos a hacer en las prácticas?

20 semanas

¿Qué vamos a hacer en las prácticas?



¿Qué vamos a hacer en las prácticas?



¿Qué vamos a hacer en las prácticas?

ROS

Intro a ROS2 | Parcial | TP final

Repaso y exploración de lo visto en teórica
+ aplicación en simulación con ROS2

Introducción a ROS

1402

ROS

1. ¿Porqué ROS?
2. ¿Qué es ROS?
3. En nuestro caso...
4. Compatibilidad
5. Tarea para la casa

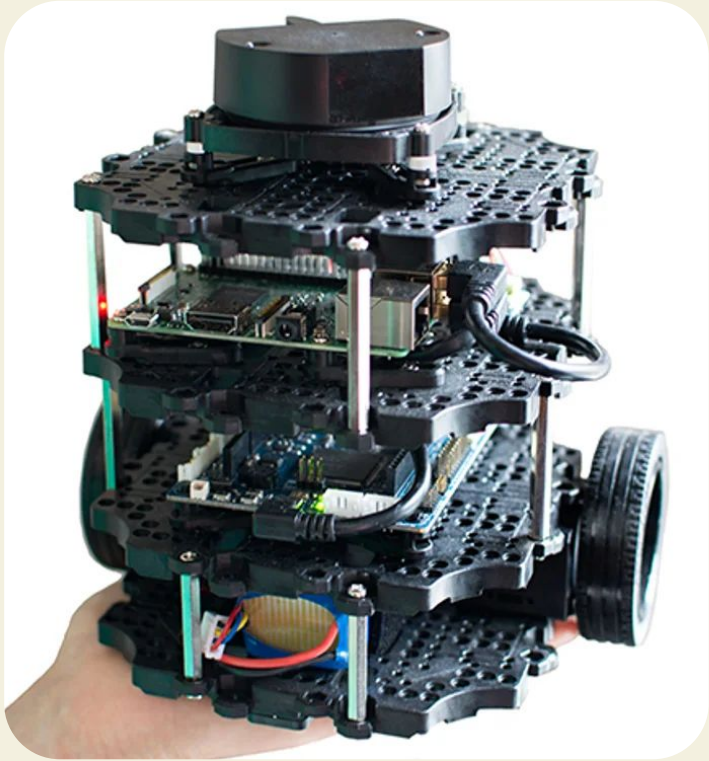
1. ¿Porqué ROS?
2. ¿Qué es ROS?
3. En nuestro caso...
4. Compatibilidad
5. Tarea para la casa

Tiene:

- Procesador/CPU
- Ruedas → capacidad de movimiento
- Algún sensor

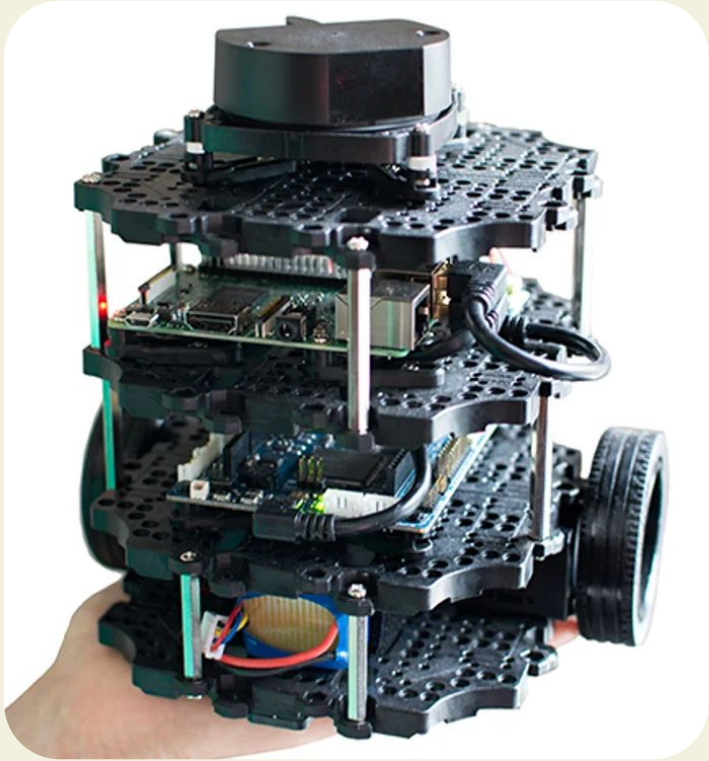
Quiero:

- Programar un algoritmo de control para que recorra un ambiente y llegue a algún destino de manera autónoma



Hago un código que en todo momento:

- Lea los datos del sensor
- Interprete lo que ve del sensor
- Decida qué hacer
- Ejecute los comandos de movimiento
- Mida que tan bien se movió en base a lo que le pedí
- Calcule el mejor camino para llegar al objetivo
- Se comunique con una PC externa para monitorear su desempeño



Hago un código que en todo momento:

- Lea los datos del sensor
- Interprete lo que ve del sensor
- Decida qué hacer
- Ejecute los comandos de movimiento
- Mida que tan bien se movió en base a lo que le pedí
- Calcule el mejor camino para llegar al objetivo
- Se comunique con una PC externa para monitorear su desempeño

Esto implica que tengo que lidiar con:

- ➔ Interrupciones por lectura de datos
- ➔ Procesar datos nuevos cada vez
- ➔ Esperar a que haya datos nuevos
- ➔ Comunicarme con el hardware
- ➔ Ante cada movimiento y lectura de datos procesar y evaluar
- ➔ Estar al tanto del movimiento en todo momento
- ➔ Networking entre PC y procesador del robot

Esto implica que tengo que lidiar con:

- ➔ Interrupciones por lectura de datos
- ➔ Procesar datos nuevos cada vez
- ➔ Esperar a que haya datos nuevos
- ➔ Comunicarme con el hardware
- ➔ Ante cada movimiento y lectura de datos procesar y evaluar
- ➔ Estar al tanto del movimiento en todo momento
- ➔ Networking entre PC y procesador del robot

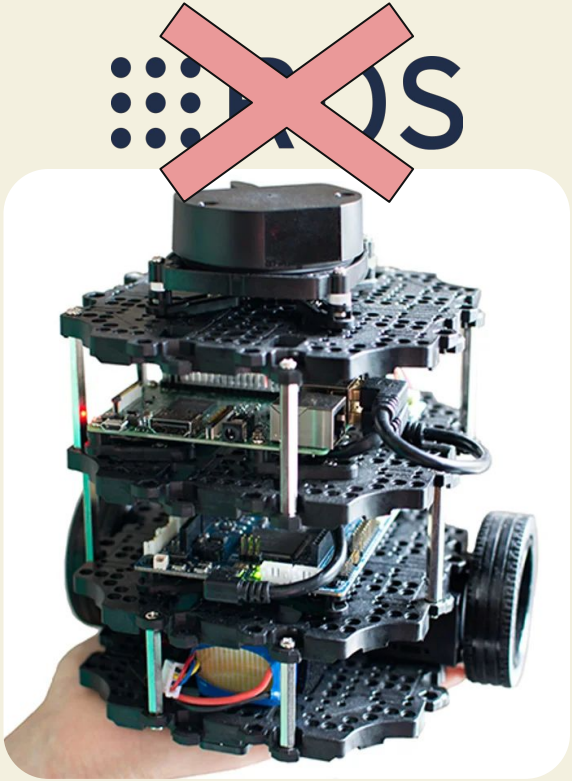
Si bajamos más a lo que implica para nuestro código ahora tenemos que lidiar con:

- Threading para manejar todos los procesos a destiempo sin bloquear el algoritmo
- Comunicación entre computadoras a través de alguna red, tambien a destiempo
- Flujo de datos entre sensores y actuadores y el procesador

1. ¿Porqué ROS?
2. **¿Qué es ROS?**
3. En nuestro caso...
4. Compatibilidad
5. Tarea para la casa

ROS

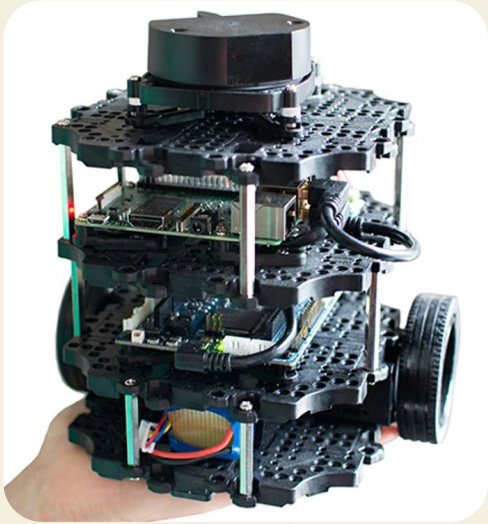
Robot Operating System



ROS

Robot Operating System

ROS



ROS

Se va a encargar de manejar:

- Todo tipo de threading para manejar procesos a destiempo sin bloquear el algoritmo
- Comunicación entre computadoras a través de alguna red, tambien a destiempo
- Flujo de datos entre sensores y actuadores y el procesador

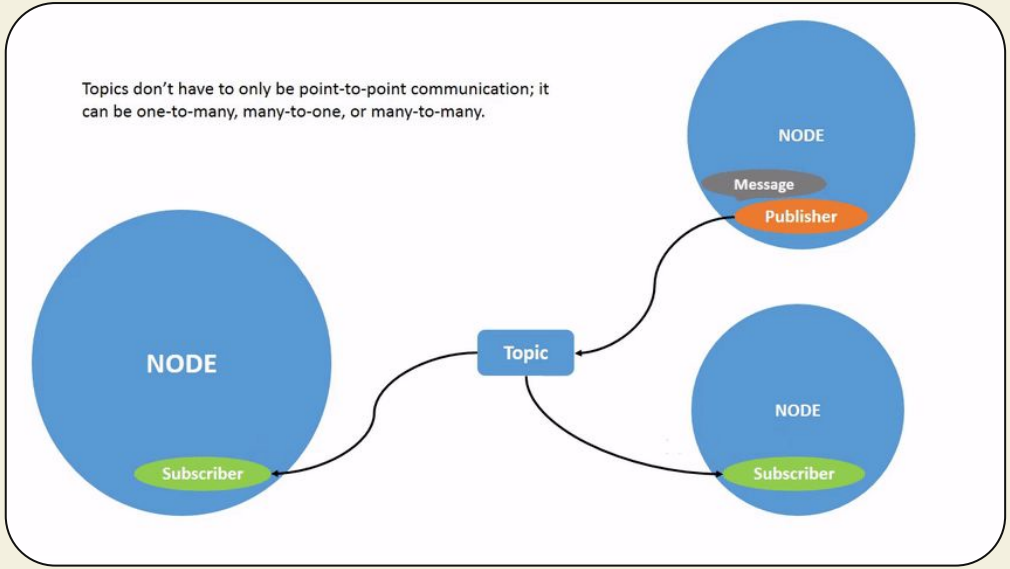
Pero además tengo:

- Software de simulación física de mi robot, sus sensores y el entorno
- Visualización de los datos de mi robot
- Grabación integrada de todos los datos de mis ensayos para una posterior evaluación o reproducción en “tiempo real”
- Paquetes pre-armados por terceros
- Aislación de paquetes para una mayor facilidad de modificaciones
- Abstracción de lenguaje (Python y C++)
- Comunidad activa de usuarios

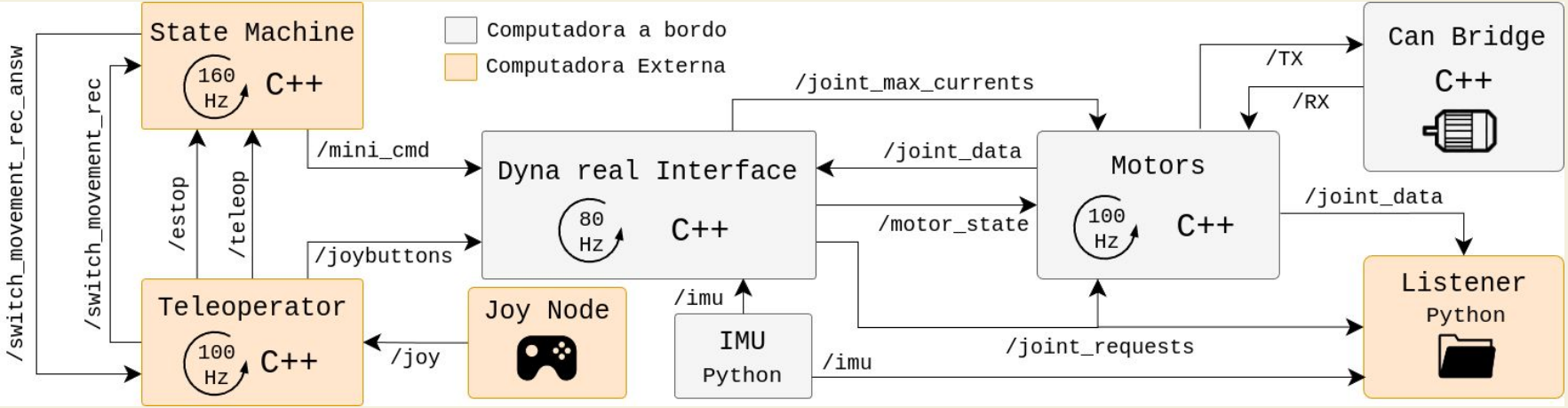
ROS

ROS: Arquitectura

- Nodo
  - Corre procesos
  - Puede publicar y suscribirse a topics
- Topics
  - Canales para mandar mensajes
- Mensajes
  - Datos que los nodos usan para comunicarse



ROS


State Machine
160 Hz
C++
Computadora a bordo
Computadora Externa
/switch_movement_rec_answ
/switch_movement_rec
/estop
/teleop
/mini_cmd
/joint_max_currents
Can Bridge
C++
/TX
/RX
Dyna real Interface
80 Hz
C++
/joint_data
Motors
100 Hz
C++
/joint_data
/motor_state
/joybuttons
Teleoperator
100 Hz
C++
Joy Node
/joy
/imu
IMU
Python
/imu
/joint_requests
Listener
Python

ROS: Arquitectura

- Nodo
  - Corre procesos
  - Puede publicar y suscribirse a topics
- Topics
  - Canales para mandar mensajes
- Mensajes
  - Datos que los nodos usan para comunicarse
- Servicios
- Acciones
- Rosbag
- Nodos composables
- QOS de mensajes
- Parámetros
- RTO
- URDF
- Capa de DDS y variantes
- Memory Allocation
- ROS2 Tracing
- Simuladores
- Mucho mucho más

Nathan Dunkelberger
5d

I'm happy to share some work that my team has been supporting! On the iMETRO team at NASA JSC, we developed a ros2_control hardware interface to support running standardized MuJoCo simulations in the ROS2 ecosystem. This work helped us create some high-fidelity simulations of our iMETRO facility, where we can test intravehicular robotics applications (see some examples in the videos below).

The ros-controls organization ended up bringing it under their domain, and just had the first official release! Feel free to check it out and contribute!

1. ¿Porqué ROS?
2. ¿Qué es ROS?
3. **En nuestro caso...**
4. Compatibilidad
5. Tarea para la casa

Versión

HUMBLE HAWKSBILL

Simulador

GAZEBO

Paquetes

TURTLEBOT3

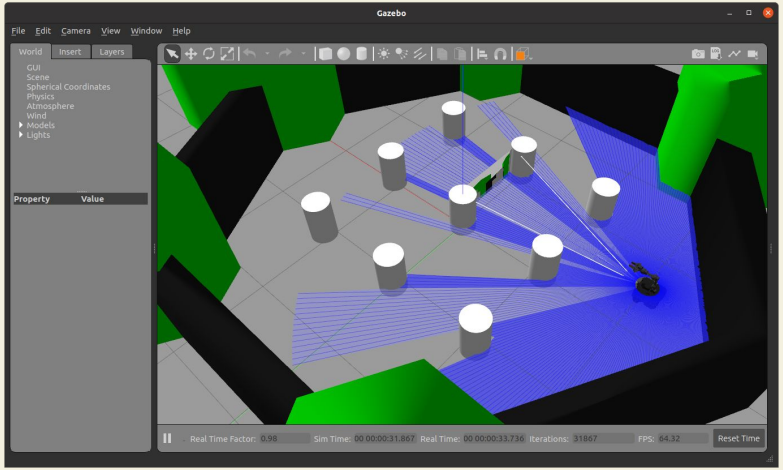
ROS2 Humble → Ojo con la versión de la documentación que están usando

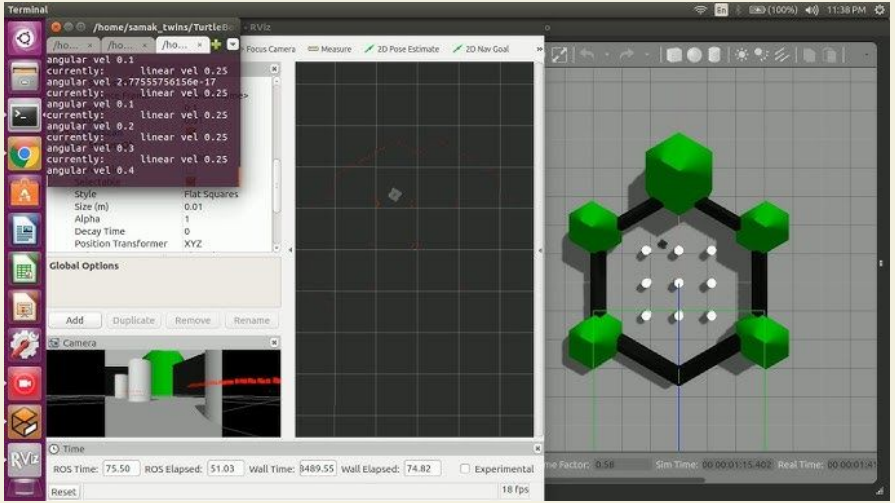
Turtlebot3 → Manual del paquete

https://docs.ros.org/en/humble/index.htm

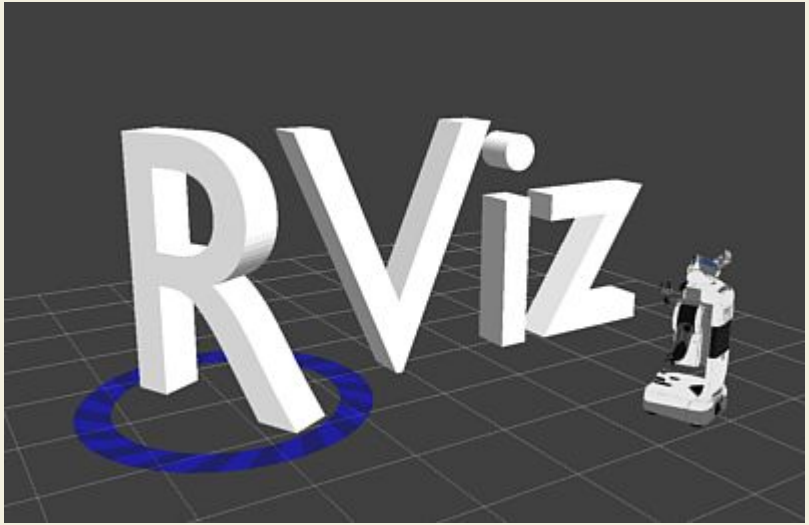
https://emanual.robotis.com/docs/en/platform/turtlebot3/simulation/

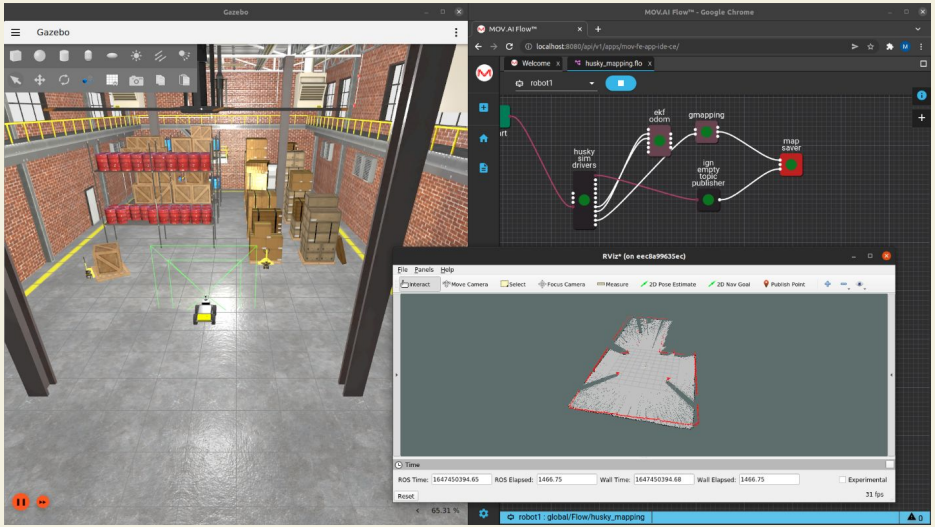
Gazebo - Simulo el robot y el entorno





RVIZ – ¿Qué ve el robot?





1. ¿Porqué ROS?
2. ¿Qué es ROS?
3. En nuestro caso...
4. **Compatibilidad**
5. Tarea para la casa

Open source...

- 100% de compatibilidad con Linux
- Especificamente Ubuntu (cada versión de ROS es compatible con algunas versiones de Ubuntu en particular)
- Open Robotics está abriéndose a nuevos OS lentamente

- Hay muchos desarrollos para trabajar ROS usando otros OS's!
- No son triviales...

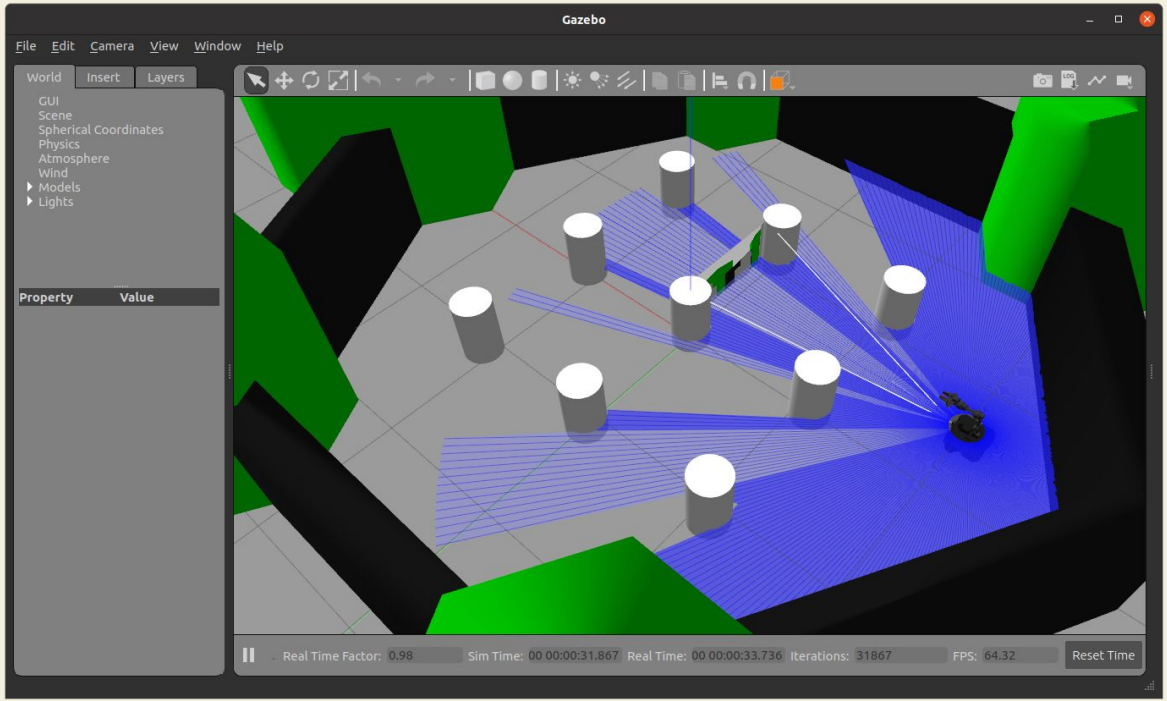
macOS

LINUX

Windows

1. ¿Porqué ROS?
2. ¿Qué es ROS?
3. En nuestro caso...
4. Compatibilidad
5. **Tarea para la casa**

Tarea para la semana que viene



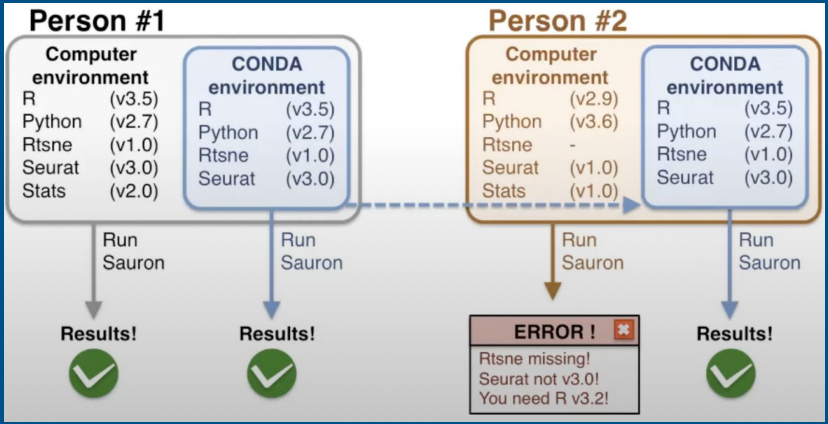
Limitantes

- Gazebo opera con el GPU
- Precisa GPU de al menos 1Gb (verificar requisito)
- La interacción con el GPU es variable según la arquitectura del microprocesador (ARM, AMD, Intel) -> cada alternativa está adaptada a arquitecturas de chip específica, siendo ARM (MacOS, chip de línea M) la menos disponible

CONDA

Robostack

- Un conjunto de paquetes Conda desarrollados por Open Robotics
- Virtual environment
- Ajusta la instalación de paquetes según OS y chip
- Comunidad muy activa en Git
- Falla mucho pero una vez instalado es altamente reproducible



Robostack

- Primero se instala el instalador de Conda, usaremos Miniforge: https://conda-forge.org/download/
- Luego se instala el ambiente Conda donde usaremos Robostack: https://robostack.github.io/GettingStarted.html
- Seguir pasos de las páginas provistas.

Robostack

- Primero se instala el instalador de Conda, usaremos Miniforge: https://conda-forge.org/download/
- Luego se instala el ambiente Conda donde usaremos Robostack: https://robostack.github.io/GettingStarted.html
- Seguir pasos de las páginas provistas.

Se puede instalar todo el ambiente de una línea de código:

mamba create -n rosenv -c conda-forge -c robostack-staging ros-humble-desktop ros-humble-turtlebot3 compilers cmake pkg-config make ninja colcon-common-extensions catkin_tools rosdep ros-humble-turtlebot3-bringup ros-humble-turtlebot3-cartographer ros-humble-turtlebot3-description ros-humble-turtlebot3-example ros-humble-turtlebot3-msgs ros-humble-turtlebot3-navigation2 ros-humble-turtlebot3-node ros-humble-turtlebot3-teleop ros-humble-turtlesim ros-humble-turtlebot3-gazebo ros2-distro-mutex==0.5.0

OjO con los breaklines al pegar

Activar el envionment:
mamba activate rosenv

Probar la instalación corriendo:
rviz2

Y luego la prueba de fuego...

export TURTLEBOT3_MODEL=burger
ros2 launch turtlebot3_gazebo empty_world.launch.py

Si inicia Gazebo y se ve el Turtlebot3 Burger ha funcionado. De lo contrario veremos de resolver el problema juntos

Linux

1. Prueben instalando con robostack (para Ubuntu 22.04 anda perfecto, Ubuntu 24 puede fallar)

2. Si falla con otra cosa que Ubuntu 22.04 y no hay solución:
   a. Instalar Ubuntu 22.04 donde prefieran (reinstalar o partición)
   b. Usan las máquinas del laboratorio
   c. Usan un docker de ros2 humble e instalan turtlebot3 a mano
   d. Se hacen amigo de alguien al que le ande

https://docs.ros.org/en/humble/How-To-Guides/Run-2-nodes-in-single-or-separate-docker-containers.html

macOS

1. Prueben instalando con robostack (si falla hablar con Bauti)

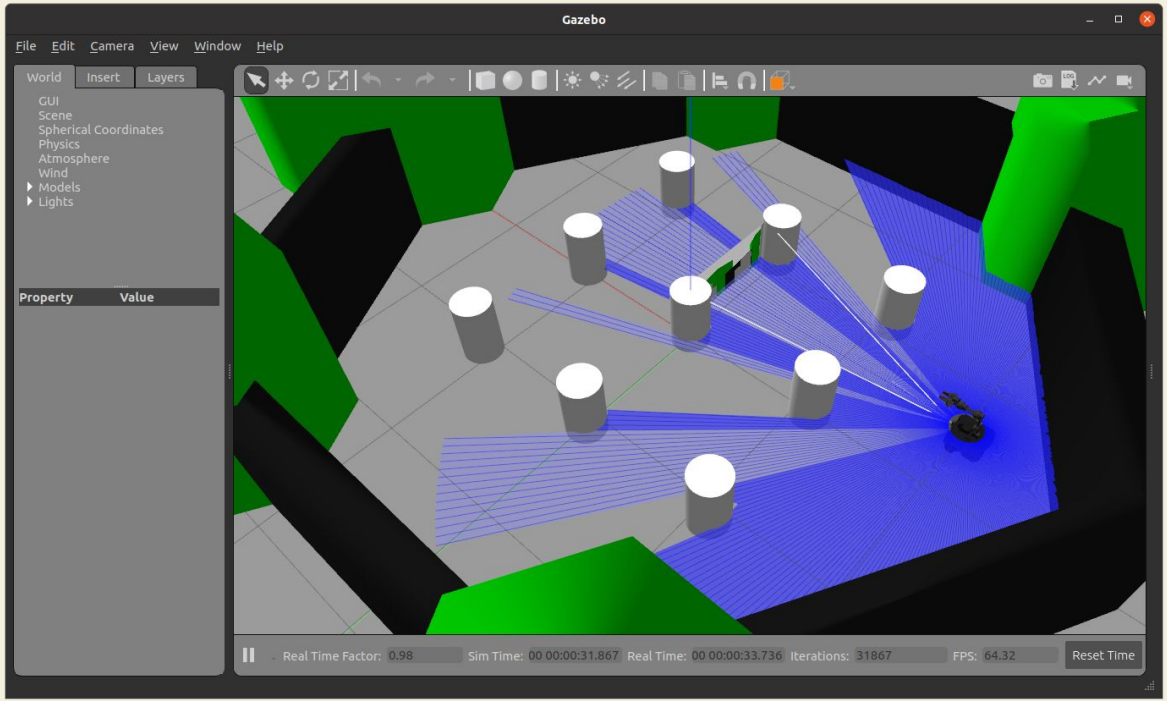
2. Si no se puede hacer andar:
   a. Usan las máquinas del laboratorio
   b. Se hacen amigo de alguien al que le ande

Windows

5 opciones…

1. Prueban con Robostack (Puede no funcionar) (Ojo que en el command prompt no es "**export** TURTLEBOT3_MODEL=burger", es "**set** TURTLEBOT3_MODEL=burger")
2. Prueban con WSL (Puede no funcionar)
3. Hacen una partición y se instalan Ubuntu
4. Usan las computadoras del laboratorio
5. Se hacen amigos de alguien a quien le ande y siempre que se hable de ROS lo ven juntos, mismo para el TP final

Tarea para la semana que viene



Extra una vez que tienen todo andando

ROS2 Crudo (Sin Robostack)

1. Si siguieron los pasos para instalar el workspace del turtlebot hacemos lo siguiente:
   a. Abrir una terminal en la carpeta del workspace
   b. cd src
   c. Mover la carpeta de “turtlebot_custom_simulation” a esta carpeta (descargan del campus y descomprimir)
   d. cd ..
   e. colcon build --symlink-install
   f. echo 'source /path/to/turtlebot3_ws/install/setup.bash' >> ~/.bashrc

Poner el path a workspace, por ejemplo, /home/usuario/turtlebot3_ws o algo asi

Ojo si tienen zsh u otra cosa en la terminal

ROS2 Crudo (Sin Robostack)

1. Si siguieron los pasos para instalar el workspace del turtlebot hacemos lo siguiente:
   a. Abrir una terminal en la carpeta del workspace
   b. cd src
   c. Mover la carpeta de “turtlebot_custom_simulation” a esta carpeta (descargan del campus y descomprimir)
   d. cd ..
   e. colcon build --symlink-install
   f. echo 'source /path/to/turtlebot3_ws/install/setup.bash' >> ~/.bashrc
   g. echo 'export TURTLEBOT3_MODEL=burger ' >> ~/.bashrc
   h. source ~/.bashrc

Si hicieron todo bien, deberían poder abrir otra terminal y correr el siguiente código debería lanzar una simulación con PISO BLANCO

```
ros2 launch turtlebot3_custom_simulation custom_simulation.launch.py
```

ROS2 en Robostack

1. Si siguieron los pasos para instalar el workspace del turtlebot hacemos lo siguiente:
   a. Mamba activate rosenv
   b. Cd && cd ws - o si no tienen un ws hecho - cd && mkdir -p ws/src && cd ws && colcon build
   c. cd src
   d. Mover la carpeta de “turtlebot_custom_simulation” a esta carpeta (descargan del campus y descomprimir)
   e. cd ..
   f. colcon build --symlink-install
   g. echo 'source /path/to/ws/install/setup.bash' >> ~/.bashrc (o .zsh, atentos: echo $SHELL)

Poner el path a workspace: cd && echo pwd

Ojo si tienen zsh u otra cosa en la terminal

ROS2 en Robostack

1. Si siguieron los pasos para instalar el workspace del turtlebot hacemos lo siguiente:
   a. Mamba activate rosenv
   b. Cd && cd ws - o si no tienen un ws hecho - cd && mkdir -p ws/src && cd ws && colcon build (only once…)
   c. cd src
   d. Mover la carpeta de “turtlebot_custom_simulation” a esta carpeta (descargan del campus y descomprimir)
   e. cd ..
   f. colcon build --symlink-install
   g. echo 'source /path/to/ws/install/setup.bash' >> ~/.bashrc (o .zsh, atentos: echo $SHELL)
   h. echo 'export TURTLEBOT3_MODEL=burger ' >> ~/.bashrc
   i. source ~/.bashrc && ros2 launch turtlebot3_custom_simulation custom_simulation.launch.py